

TP Parallélisme - Exercice 2

Gabriel Mazet - L3 Informatique

Somme des éléments d'une matrice avec OpenMP.

1. Énoncé

On veut calculer la somme des éléments d'une matrice d'entiers avec 4 versions :

1. Version séquentielle (V1).
2. Version parallèle avec sommes partielles dans un tableau, somme finale faite par un seul thread (V2).
3. Version parallèle avec sommes partielles, mais chaque thread ajoute sa somme à la globale en exclusion mutuelle (V3).
4. Version parallèle avec `reduction` (V4).

Matrice 10000×10000 remplie de 1 (pour éviter les débordements et vérifier le résultat : on doit obtenir 10^8). Compilation sans optimisation : `gcc -O0 -fopenmp exo2.c -o exo2`.

2. Environnement de test

- **CPU** : Intel Core i7-8650U, 4 cœurs physiques / 8 threads logiques (HyperThreading).
- **Cache** : L1d = 32 Kio, ligne de cache = 64 octets, L2 = 256 Kio, L3 = 8 Mio.
- **Mesures** : `omp_get_wtime()` , moyenne sur 3 exécutions.

La matrice fait $10000 \times 10000 \times 4$ octets ≈ 400 Mo, donc largement plus grosse que le L3. À chaque parcours on tape la RAM, le programme est *memory-bound*.

3. Ce que j'ai fait pour paralléliser

V1 - Séquentielle

Double boucle classique, accumulateur en `long long` (sinon dépassement avec un `int`). Sert de référence pour comparer.

V2 - Sommes partielles dans un tableau

J'alloue un tableau `sommesPartielles` de taille `nb_thread` . Dans la région parallèle, chaque thread calcule sa somme dans une variable locale (`sommeLocale`) puis écrit le résultat à l'index `omp_get_thread_num()` . Une fois la région fermée, le thread principal fait la somme du tableau.

Choix : j'utilise une variable locale au lieu d'écrire directement dans `sommesPartielles[idThread]` à chaque itération. Sinon on aurait du *false sharing* : 8 `long long` tiennent dans une ligne de 64 octets, donc tous les threads se battraient pour la même ligne de cache.

V3 - Exclusion mutuelle (`critical`)

Même idée que V2 mais à la fin chaque thread ajoute sa somme locale à `sommeTotale` (variable partagée) dans un `#pragma omp critical`. Sans la section critique on aurait une *data race* : deux threads peuvent lire la même valeur de `sommeTotale`, ajouter leur part, puis l'un écrase l'autre → on perd des additions.

Petite optimisation : j'ai mis `nowait` sur le `#pragma omp for`. Comme ça, dès qu'un thread finit son chunk, il rentre direct dans la section critique au lieu d'attendre la barrière implicite. Note : `atomic` aurait aussi marché vu que c'est juste une addition.

V4 - Réduction

Une seule ligne avec `reduction(+:sommeTotale)`. OpenMP s'occupe de tout : il crée une copie privée par thread et combine à la fin. C'est la version la plus simple.

Difficultés rencontrées

- Au début j'avais oublié le `long long` sur la somme et je tombais sur des valeurs négatives à cause du débordement (10^8 rentre dans un `int` de justesse mais avec une matrice plus grande ça casserait).
- J'ai hésité entre `atomic` et `critical` pour V3. `atomic` est plus rapide en théorie mais comme l'opération a lieu une seule fois par thread (pas dans la boucle), la différence est invisible.
- Penser à initialiser `sommeTotale = 0` avant chaque appel - sinon on accumule les résultats des appels précédents.

4. Résultats

Temps de la version séquentielle (V1) : **0.393 s**.

Threads	V2 Partiel (s)	V3 Mutex (s)	V4 Reduction (s)
1	0.397	0.399	0.396
2	0.200	0.199	0.198
4	0.118	0.122	0.123
8	0.096	0.087	0.087

Threads	V2 Partiel (s)	V3 Mutex (s)	V4 Reduction (s)
16	0.087	0.086	0.088

5. Commentaires sur les résultats

Les 3 versions parallèles donnent quasi le même temps

C'est ce qui m'a le plus surpris au début. En fait, comme on compile en `-O0`, le compilateur ne vectorise pas et n'optimise pas les boucles. Du coup le coût dominant c'est l'addition + l'accès mémoire, et ces trois opérations sont identiques dans V2/V3/V4. Le seul truc qui change c'est comment on combine les sommes partielles à la fin, et ça représente seulement `nb_thread` additions face à 10^8 additions de boucle. C'est complètement négligeable.

Si on activait `-O2`, je pense que V4 prendrait plus d'avance car la `reduction` permet une vectorisation propre (l'accumulateur privé devient un registre SIMD).

Le scaling sature autour de 4 threads

- **1 → 2 threads** : on divise le temps par presque 2 ($\times 1.97$). Très bon scaling.
- **2 → 4 threads** : $\times 1.65$ seulement. On commence à être limité par la bande passante mémoire (matrice trop grande pour le cache).
- **4 → 8 threads** : $\times 1.40$. Là on passe sur l'HyperThreading. Le gain vient du fait que pendant qu'un thread attend la mémoire, l'autre thread du même cœur peut bosser.
- **8 → 16 threads** : aucun gain (parfois même un peu plus lent). Logique : on n'a que 8 threads logiques, demander 16 ne sert à rien à part rajouter du coût d'ordonnancement.

Pourquoi le mutex de V3 ne pénalise pas

Je pensais que la section critique allait faire perdre du temps mais en fait non. Avec `nowait`, les `nb_thread` threads font leurs `nb_thread` additions en série, ce qui prend quelques microsecondes face à 100 ms de calcul. Le mutex serait dramatique si on l'utilisait *dans* la boucle (genre `sommeTotale += matrice[i*c+j]` en critical) - là on sérialise tout et on tue le parallélisme.

Lequel choisir ?

Pour ce problème en `-O0` les trois se valent. Mais V4 (reduction) reste mon choix par défaut :

- Une seule ligne, plus simple à lire et à maintenir.
- Pas d'allocation auxiliaire (V2 doit faire `calloc / free`).
- Le combine final est fait en arbre par OpenMP, donc ça scale mieux quand on a beaucoup de threads.

6. Vérification

Toutes les versions retournent `somme = 100000000 = 108`, ce qui est bien le résultat attendu ($10000 \times 10000 \times 1$). Pas de débordement grâce au `long long`.